Prims algorthim

```
#include <iostream>
using namespace std;

#define V 100
#define INF 9999

int n, e;
int graph[V][V];

void input() {

    cout << "Enter number of vertices: ";
    cin >> n;

    cout << "Enter number of edges: ";
    cin >> e;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            graph[i][j] = 0;
        }
    }

    cout << "Enter edges (u v weight):\n";
    for (int i = 0; i < e; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        graph[u][v] = w;
```

```
        graph[v][u] = w;
    }
}


void primMST() {

    int visited[V] = {0};
    visited[0] = 1;

    int edgeCount = 0;
    int totalCost = 0;

    cout << "\nEdges in Minimum Spanning Tree:\n";

    while (edgeCount < n - 1) {

        int min = INF;
        int a = -1, b = -1;



        for (int i = 0; i < n; i++) {
            if (visited[i]) {
                for (int j = 0; j < n; j++) {
                    if (!visited[j] && graph[i][j] != 0) {
                        if (graph[i][j] < min) {
                            min = graph[i][j];
                            a = i;
                            b = j;
```

```
                }
            }
        }
    }
    }

    cout << a << " - " << b << " : " << min << endl;

    visited[b] = 1;
    totalCost += min;
    edgeCount++;
  }

  cout << "\nTotal MST Cost = " << totalCost << endl;
}


int main() {

  input();
  primMST();

  return 0;
}
```

Kruskal's algorithm

```
#include <iostream>
using namespace std;

#define V 100
#define E 100



struct Edge {
   int u, v, w;
};

Edge edges[E];
int parent[V];



int findParent(int i) {
   while (parent[i] != i)
      i = parent[i];
   return i;
}



void unionSet(int a, int b) {
   parent[b] = a;
}

int main() {
```

```
int n, e;
cout << "Enter number of vertices: ";
cin >> n;

cout << "Enter number of edges: ";
cin >> e;



cout << "Enter edges (u v weight):\n";
for (int i = 0; i < e; i++) {
    cin >> edges[i].u >> edges[i].v >> edges[i].w;
}



for (int i = 0; i < e - 1; i++) {
    for (int j = 0; j < e - i - 1; j++) {
        if (edges[j].w > edges[j + 1].w) {
            Edge temp = edges[j];
            edges[j] = edges[j + 1];
            edges[j + 1] = temp;
        }
    }
}



for (int i = 0; i < n; i++)
    parent[i] = i;
```

```
    int cost = 0;
    int count = 0;

    cout << "\nEdges in Minimum Spanning Tree:\n";

    for (int i = 0; i < e; i++) {

        int u = findParent(edges[i].u);
        int v = findParent(edges[i].v);

        // if no cycle
        if (u != v) {
            cout << edges[i].u << " - " << edges[i].v
                << " : " << edges[i].w << endl;

            unionSet(u, v);
            cost += edges[i].w;
            count++;
        }

        if (count == n - 1)
            break;
    }

    cout << "\nTotal MST Cost = " << cost << endl;

    return 0;
}
```

BFS

```
#include <iostream>
using namespace std;

#define MAX 100

int N, M;



int qx[MAX*MAX], qy[MAX*MAX];
int front = 0, rear = 0;

int dx[] = {1, -1, 0, 0};
int dy[] = {0, 0, 1, -1};



bool isSafe(int maze[MAX][MAX], int visited[MAX][MAX], int x, int y) {
    return (x >= 0 && x < N &&
            y >= 0 && y < M &&
            maze[x][y] == 0 &&
            visited[x][y] == 0);
}

void bfs(int maze[MAX][MAX], int sx, int sy, int gx, int gy) {

    int visited[MAX][MAX] = {0};
```

```
int parentX[MAX][MAX], parentY[MAX][MAX];


front = rear = 0;
qx[rear] = sx;
qy[rear] = sy;
rear++;

visited[sx][sy] = 1;
parentX[sx][sy] = -1;
parentY[sx][sy] = -1;

bool found = false;

while (front < rear) {

    int x = qx[front];
    int y = qy[front];
    front++;


    if (x == gx && y == gy) {
        found = true;
        break;
    }


    for (int i = 0; i < 4; i++) {
```

```
        int nx = x + dx[i];
        int ny = y + dy[i];

        if (isSafe(maze, visited, nx, ny)) {
            qx[rear] = nx;
            qy[rear] = ny;
            rear++;

            visited[nx][ny] = 1;



            parentX[nx][ny] = x;
            parentY[nx][ny] = y;
        }
    }
}



if (found) {
    cout << "\nPath (start to goal):\n";

    int pathX[MAX*MAX], pathY[MAX*MAX];
    int len = 0;

    int x = gx, y = gy;



    while (x != -1 && y != -1) {
        pathX[len] = x;
```

```
            pathY[len] = y;
            len++;

            int px = parentX[x][y];
            int py = parentY[x][y];

            x = px;
            y = py;
        }



        for (int i = len - 1; i >= 0; i--) {
            cout << "(" << pathX[i] << "," << pathY[i] << ") ";
        }

        cout << "\nPath Found\n";
    } else {
        cout << "\nNo Path Found\n";
    }
}

int main() {

    int maze[MAX][MAX];
    int sx, sy, gx, gy;

    cout << "Enter rows and columns: ";
    cin >> N >> M;
```

```
    cout << "Enter maze (0=open, 1=blocked):\n";
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < M; j++) {
            cin >> maze[i][j];
        }
    }

    cout << "Enter start (row col): ";
    cin >> sx >> sy;

    cout << "Enter goal (row col): ";
    cin >> gx >> gy;

    if (maze[sx][sy] == 1 || maze[gx][gy] == 1) {
        cout << "Invalid start or goal\n";
    } else {
        bfs(maze, sx, sy, gx, gy);
    }

    return 0;
}
```

Dfs

```
#include <iostream>
using namespace std;

#define MAX 100

int N, M;
int maze[MAX][MAX];
```

```
int visited[MAX][MAX];


int pathX[MAX*MAX], pathY[MAX*MAX];
int pathLen = 0;


int dx[] = {1, -1, 0, 0};
int dy[] = {0, 0, 1, -1};


bool isSafe(int x, int y) {
    return (x >= 0 && x < N &&
            y >= 0 && y < M &&
            maze[x][y] == 0 &&
            !visited[x][y]);
}


bool dfs(int x, int y, int gx, int gy) {

    visited[x][y] = 1;


    pathX[pathLen] = x;
    pathY[pathLen] = y;
    pathLen++;
```

```
    if (x == gx && y == gy)
        return true;



    for (int i = 0; i < 4; i++) {
        int nx = x + dx[i];
        int ny = y + dy[i];

        if (isSafe(nx, ny)) {
            if (dfs(nx, ny, gx, gy))
                return true;
        }
    }



    pathLen--;
    return false;
}

int main() {

    int sx, sy, gx, gy;

    cout << "Enter rows and columns: ";
    cin >> N >> M;

    cout << "Enter maze (0=open, 1=blocked):\n";
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < M; j++) {
```

```
            cin >> maze[i][j];
            visited[i][j] = 0;
        }
    }

    cout << "Enter start (row col): ";
    cin >> sx >> sy;

    cout << "Enter goal (row col): ";
    cin >> gx >> gy;

    if (maze[sx][sy] == 1 || maze[gx][gy] == 1) {
        cout << "Invalid start or goal\n";
    }
    else if (dfs(sx, sy, gx, gy)) {

        cout << "\nPath (start to goal):\n";
        for (int i = 0; i < pathLen; i++) {
            cout << "(" << pathX[i] << "," << pathY[i] << ") ";
        }
        cout << "\nPath Found\n";
    }
    else {
        cout << "\nNo Path Found\n";
    }

    return 0;
}
```

B-tree

```
#include<iostream>
using namespace std;

#define MAX 2

struct Node {
   int keys[3];
   Node* child[4];
   int count;
   bool leaf;
};

Node* root = NULL;

Node* createNode() {
   Node* t = new Node;
   t->count = 0;
   t->leaf = true;
   for(int i=0;i<4;i++) t->child[i] = NULL;
   return t;
}

void insertNode(Node* node, int val) {
   int i = node->count - 1;
   while(i >= 0 && node->keys[i] > val) {
      node->keys[i+1] = node->keys[i];
      i--;
   }
   node->keys[i+1] = val;
```

```
    node->count++;
}

void insert(int val) {
    if(root == NULL) {
        root = createNode();
        root->keys[0] = val;
        root->count = 1;
        return;
    }
    insertNode(root, val);
}

void levelOrder() {
    if(root == NULL) return;

    Node* q[100];
    int front = 0, rear = 0;

    q[rear++] = root;

    while(front < rear) {
        Node* temp = q[front++];

        for(int i=0;i<temp->count;i++)
            cout << temp->keys[i] << " ";
        cout << "| ";

        for(int i=0;i<=temp->count;i++) {
```

```
            if(temp->child[i] != NULL)
                q[rear++] = temp->child[i];
        }
    }
    cout << endl;
}

void deleteKey(int val) {
    if(root == NULL) return;

    for(int i=0;i<root->count;i++) {
        if(root->keys[i] == val) {
            for(int j=i;j<root->count-1;j++)
                root->keys[j] = root->keys[j+1];
            root->count--;
            break;
        }
    }
}

int main() {
    int n, val, choice;

    while(1) {
        cout << "\n1.Insert\n2.Display\n3.Delete\n4.Exit\n";
        cin >> choice;

        if(choice == 1) {
            cout << "Enter value: ";
```

```
            cin >> val;
            insert(val);
        }
        else if(choice == 2) {
            levelOrder();
        }
        else if(choice == 3) {
            cout << "Enter value to delete: ";
            cin >> val;
            deleteKey(val);
        }
        else {
            break;
        }
    }
return 0;
}
```

Redbalck-tree

```
#include<iostream>
using namespace std;

enum Color {RED, BLACK};

struct Node {
    string name;
    Color color;
    Node *left, *right, *parent;
};
```

```
Node *root = NULL;

Node* createNode(string name) {
   Node* temp = new Node;
   temp->name = name;
   temp->color = RED;
   temp->left = temp->right = temp->parent = NULL;
   return temp;
}

void rotateLeft(Node* &root, Node* &pt) {
   Node* pt_right = pt->right;
   pt->right = pt_right->left;

   if(pt->right != NULL)
      pt->right->parent = pt;

   pt_right->parent = pt->parent;

   if(pt->parent == NULL)
      root = pt_right;
   else if(pt == pt->parent->left)
      pt->parent->left = pt_right;
   else
      pt->parent->right = pt_right;

   pt_right->left = pt;
   pt->parent = pt_right;
}
```

```
void rotateRight(Node* &root, Node* &pt) {
   Node* pt_left = pt->left;
   pt->left = pt_left->right;

   if(pt->left != NULL)
      pt->left->parent = pt;

   pt_left->parent = pt->parent;

   if(pt->parent == NULL)
      root = pt_left;
   else if(pt == pt->parent->left)
      pt->parent->left = pt_left;
   else
      pt->parent->right = pt_left;

   pt_left->right = pt;
   pt->parent = pt_left;
}

void fixInsert(Node* &root, Node* &pt) {
   Node* parent_pt = NULL;
   Node* grand_parent_pt = NULL;

   while((pt != root) && (pt->color != BLACK) &&
        (pt->parent->color == RED)) {

      parent_pt = pt->parent;
```

```
grand_parent_pt = pt->parent->parent;

if(parent_pt == grand_parent_pt->left) {
    Node* uncle_pt = grand_parent_pt->right;

    if(uncle_pt != NULL && uncle_pt->color == RED) {
        grand_parent_pt->color = RED;
        parent_pt->color = BLACK;
        uncle_pt->color = BLACK;
        pt = grand_parent_pt;
    }
    else {
        if(pt == parent_pt->right) {
            rotateLeft(root, parent_pt);
            pt = parent_pt;
            parent_pt = pt->parent;
        }
        rotateRight(root, grand_parent_pt);
        Color t = parent_pt->color;
        parent_pt->color = grand_parent_pt->color;
        grand_parent_pt->color = t;
        pt = parent_pt;
    }
}
else {
    Node* uncle_pt = grand_parent_pt->left;

    if((uncle_pt != NULL) && (uncle_pt->color == RED)) {
        grand_parent_pt->color = RED;
```

```
            parent_pt->color = BLACK;
            uncle_pt->color = BLACK;
            pt = grand_parent_pt;
        }
        else {
            if(pt == parent_pt->left) {
                rotateRight(root, parent_pt);
                pt = parent_pt;
                parent_pt = pt->parent;
            }
            rotateLeft(root, grand_parent_pt);
            Color t = parent_pt->color;
            parent_pt->color = grand_parent_pt->color;
            grand_parent_pt->color = t;
            pt = parent_pt;
        }
    }
  }
  root->color = BLACK;
}

void insert(string name) {
  Node* pt = createNode(name);

  Node* y = NULL;
  Node* x = root;

  while(x != NULL) {
    y = x;
```

```
        if(pt->name < x->name)
            x = x->left;
        else
            x = x->right;
    }

    pt->parent = y;

    if(y == NULL)
        root = pt;
    else if(pt->name < y->name)
        y->left = pt;
    else
        y->right = pt;

    if(pt->parent == NULL) {
        pt->color = BLACK;
        return;
    }

    if(pt->parent->parent == NULL)
        return;

    fixInsert(root, pt);
}

Node* search(Node* root, string key) {
    if(root == NULL || root->name == key)
        return root;
```

```
    if(key < root->name)
        return search(root->left, key);
    return search(root->right, key);
}

Node* minValue(Node* node) {
    while(node->left != NULL)
        node = node->left;
    return node;
}

Node* deleteNode(Node* root, string key) {
    if(root == NULL) return root;

    if(key < root->name)
        root->left = deleteNode(root->left, key);
    else if(key > root->name)
        root->right = deleteNode(root->right, key);
    else {
        if(root->left == NULL) {
            Node* temp = root->right;
            delete root;
            return temp;
        }
        else if(root->right == NULL) {
            Node* temp = root->left;
            delete root;
            return temp;
        }
```

```
        Node* temp = minValue(root->right);
        root->name = temp->name;
        root->right = deleteNode(root->right, temp->name);
    }
    return root;
}

void inorder(Node* root) {
    if(root == NULL) return;
    inorder(root->left);
    cout << root->name << "(" << (root->color==RED?"R":"B") << ") ";
    inorder(root->right);
}

int main() {
    int ch;
    string name;

    while(1) {
        cout << "\n1.Insert\n2.Delete\n3.Display\n4.Exit\n";
        cin >> ch;

        if(ch == 1) {
            cout << "Enter directory name: ";
            cin >> name;
            insert(name);
        }
        else if(ch == 2) {
```

```
        cout << "Enter directory to delete: ";
        cin >> name;
        root = deleteNode(root, name);
    }
    else if(ch == 3) {
        inorder(root);
        cout << endl;
    }
    else break;
  }
  return 0;
}
```

Dijkstra'

```
#include<iostream>
using namespace std;

#define MAX 100
#define INF 99999

int main() {

  int n;
  cout << "Enter number of locations: ";
  cin >> n;

  int graph[MAX][MAX];

  cout << "Enter adjacency matrix (0 if no path):\n";
```

```
for(int i=0;i<n;i++) {
    for(int j=0;j<n;j++) {
        cin >> graph[i][j];
        if(graph[i][j] == 0 && i != j)
            graph[i][j] = INF;
    }
}

int src;
cout << "Enter source location: ";
cin >> src;

int dist[MAX];
bool visited[MAX];

for(int i=0;i<n;i++) {
    dist[i] = INF;
    visited[i] = false;
}

dist[src] = 0;

for(int count=0; count<n-1; count++) {

    int min = INF, u = -1;

    for(int i=0;i<n;i++) {
        if(!visited[i] && dist[i] < min) {
            min = dist[i];
```

```
            u = i;
        }
    }

    visited[u] = true;

    for(int v=0; v<n; v++) {
        if(!visited[v] && graph[u][v] != INF &&
           dist[u] + graph[u][v] < dist[v]) {
            dist[v] = dist[u] + graph[u][v];
        }
    }
  }

  cout << "\nShortest distances from source:\n";
  for(int i=0;i<n;i++) {
    cout << "To " << i << " = " << dist[i] << endl;
  }

  return 0;
}
```

AVL

```
#include <iostream>
using namespace std;

struct Node {
  int player_id;
  int score;
  Node* left;
```

```
    Node* right;
    int height;
};

int height(Node* n) {
    return (n == NULL) ? 0 : n->height;
}

int max(int a, int b) {
    return (a > b) ? a : b;
}

Node* newNode(int player_id, int score) {
    Node* node = new Node();
    node->player_id = player_id;
    node->score = score;
    node->left = node->right = NULL;
    node->height = 1;
    return node;
}

Node* rightRotate(Node* y) {
    Node* x = y->left;
    Node* T2 = x->right;

    x->right = y;
    y->left = T2;

    y->height = max(height(y->left), height(y->right)) + 1;
```

```
    x->height = max(height(x->left), height(x->right)) + 1;

    return x;
}

Node* leftRotate(Node* x) {
    Node* y = x->right;
    Node* T2 = y->left;

    y->left = x;
    x->right = T2;

    x->height = max(height(x->left), height(x->right)) + 1;
    y->height = max(height(y->left), height(y->right)) + 1;

    return y;
}

int getBalance(Node* n) {
    return (n == NULL) ? 0 : height(n->left) - height(n->right);
}

Node* insert(Node* node, int player_id, int score) {
    if (node == NULL)
        return newNode(player_id, score);

    if (score < node->score)
        node->left = insert(node->left, player_id, score);
    else if (score > node->score)
```

```
    node->right = insert(node->right, player_id, score);
else {
    if (player_id < node->player_id)
        node->left = insert(node->left, player_id, score);
    else
        node->right = insert(node->right, player_id, score);
}

node->height = 1 + max(height(node->left), height(node->right));

int balance = getBalance(node);

if (balance > 1 && score < node->left->score)
    return rightRotate(node);

if (balance < -1 && score > node->right->score)
    return leftRotate(node);

if (balance > 1 && score > node->left->score) {
    node->left = leftRotate(node->left);
    return rightRotate(node);
}

if (balance < -1 && score < node->right->score) {
    node->right = rightRotate(node->right);
    return leftRotate(node);
}

return node;
```

```
}

Node* minValueNode(Node* node) {
    Node* current = node;
    while (current->left != NULL)
        current = current->left;
    return current;
}

Node* deleteNode(Node* root, int player_id, int score) {
    if (root == NULL) return root;

    if (score < root->score)
        root->left = deleteNode(root->left, player_id, score);
    else if (score > root->score)
        root->right = deleteNode(root->right, player_id, score);
    else if (player_id != root->player_id) {
        root->left = deleteNode(root->left, player_id, score);
        root->right = deleteNode(root->right, player_id, score);
    }
    else {
        if (root->left == NULL || root->right == NULL) {
            Node* temp = root->left ? root->left : root->right;

            if (temp == NULL) {
                temp = root;
                root = NULL;
            } else {
                *root = *temp;
```

```
        }
        delete temp;
    }
    else {
        Node* temp = minValueNode(root->right);
        root->player_id = temp->player_id;
        root->score = temp->score;
        root->right = deleteNode(root->right, temp->player_id, temp->score);
    }
}

if (root == NULL) return root;

root->height = 1 + max(height(root->left), height(root->right));

int balance = getBalance(root);

if (balance > 1 && getBalance(root->left) >= 0)
    return rightRotate(root);

if (balance > 1 && getBalance(root->left) < 0) {
    root->left = leftRotate(root->left);
    return rightRotate(root);
}

if (balance < -1 && getBalance(root->right) <= 0)
    return leftRotate(root);

if (balance < -1 && getBalance(root->right) > 0) {
```

```
        root->right = rightRotate(root->right);
        return leftRotate(root);
    }

    return root;
}

void leaderboard(Node* root) {
    if (root != NULL) {
        leaderboard(root->right);
        cout << "Player ID: " << root->player_id
            << " | Score: " << root->score << endl;
        leaderboard(root->left);
    }
}

int main() {
    Node* root = NULL;
    int choice, id, score;

    while (true) {
        cout << "\n--- Multiplayer Game System ---\n";
        cout << "1. Player Registration\n";
        cout << "2. Display Leaderboard\n";
        cout << "3. Remove Player\n";
        cout << "4. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;
```

```
        switch (choice) {
            case 1:
                cin >> id >> score;
                root = insert(root, id, score);
                cout << "Player Registered!\n";
                break;

            case 2:
                leaderboard(root);
                break;

            case 3:
                cin >> id >> score;
                root = deleteNode(root, id, score);
                cout << "Player Removed!\n";
                break;

            case 4:
                return 0;

            default:
                cout << "Invalid Choice!\n";
        }
    }
}
```

BST dictionary

```
#include <iostream>
#include <queue>
using namespace std;
```

```
struct Node {
   string key, value;
   Node* left;
   Node* right;
};

Node* createNode(string key, string value) {
   Node* temp = new Node();
   temp->key = key;
   temp->value = value;
   temp->left = temp->right = NULL;
   return temp;
}

Node* insert(Node* root, string key, string value) {
   if (root == NULL)
      return createNode(key, value);

   if (key < root->key)
      root->left = insert(root->left, key, value);
   else if (key > root->key)
      root->right = insert(root->right, key, value);
   else
      root->value = value;

   return root;
}
```

```
Node* minValueNode(Node* root) {
    Node* curr = root;
    while (curr && curr->left != NULL)
        curr = curr->left;
    return curr;
}

Node* deleteNode(Node* root, string key) {
    if (root == NULL) return root;

    if (key < root->key)
        root->left = deleteNode(root->left, key);
    else if (key > root->key)
        root->right = deleteNode(root->right, key);
    else {
        if (root->left == NULL) {
            Node* temp = root->right;
            delete root;
            return temp;
        }
        else if (root->right == NULL) {
            Node* temp = root->left;
            delete root;
            return temp;
        }

        Node* temp = minValueNode(root->right);
        root->key = temp->key;
        root->value = temp->value;
```

```
        root->right = deleteNode(root->right, temp->key);
    }
    return root;
}

void search(Node* root, string key) {
    if (root == NULL) {
        cout << "Word not found\n";
        return;
    }

    if (key == root->key)
        cout << "Found: " << root->key << " -> " << root->value << endl;
    else if (key < root->key)
        search(root->left, key);
    else
        search(root->right, key);
}

void inorder(Node* root) {
    if (root != NULL) {
        inorder(root->left);
        cout << root->key << " -> " << root->value << endl;
        inorder(root->right);
    }
}

Node* mirror(Node* root) {
    if (root == NULL) return NULL;
```

```
    Node* temp = root->left;
    root->left = mirror(root->right);
    root->right = mirror(temp);

    return root;
}

Node* copyTree(Node* root) {
    if (root == NULL) return NULL;

    Node* newNode = createNode(root->key, root->value);
    newNode->left = copyTree(root->left);
    newNode->right = copyTree(root->right);

    return newNode;
}

void levelOrder(Node* root) {
    if (root == NULL) return;

    queue<Node*> q;
    q.push(root);

    while (!q.empty()) {
        Node* curr = q.front();
        q.pop();

        cout << curr->key << " -> " << curr->value << endl;
```

```
        if (curr->left != NULL) q.push(curr->left);
        if (curr->right != NULL) q.push(curr->right);
    }
}

int main() {
    Node* root = NULL;
    Node* copyRoot = NULL;
    int choice;
    string key, value;

    while (true) {
        cout << "\n--- Dictionary BST ---\n";
        cout << "1. Insert Word\n";
        cout << "2. Delete Word\n";
        cout << "3. Search Word\n";
        cout << "4. Display Dictionary (Inorder)\n";
        cout << "5. Mirror Dictionary\n";
        cout << "6. Copy Dictionary\n";
        cout << "7. Display Copy\n";
        cout << "8. Level Order Display\n";
        cout << "9. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                cin >> key >> value;
```

```
    root = insert(root, key, value);
    break;

case 2:
    cin >> key;
    root = deleteNode(root, key);
    break;

case 3:
    cin >> key;
    search(root, key);
    break;

case 4:
    inorder(root);
    break;

case 5:
    root = mirror(root);
    cout << "Tree Mirrored\n";
    break;

case 6:
    copyRoot = copyTree(root);
    cout << "Tree Copied\n";
    break;

case 7:
    inorder(copyRoot);
```

```
            break;

        case 8:
            levelOrder(root);
            break;

        case 9:
            return 0;

        default:
            cout << "Invalid Choice\n";
    }
  }
}
```